

Lab Experiment: Prime & Palindrome Number Checker

Implementation of Static Utility Methods and Logical Verification in Java

Course: Programming in Java Lab

Language: Java (JDK 8+)

Document Code: LAB-PRG-02

1. Objective

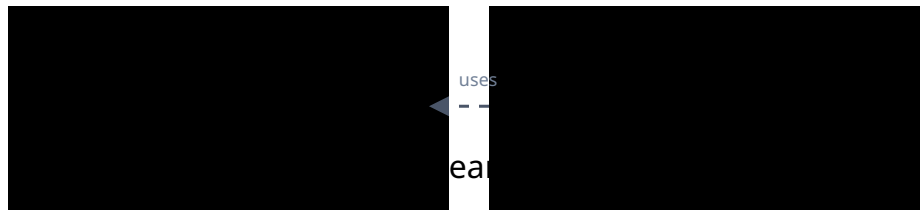
To design and implement a Java program (LabProgram2) that evaluates whether a user-input integer is a prime number, a palindrome number, both, or neither. Key technical objectives include:

- Static Utility Methods:** Encapsulating mathematical checking functions (isPrime and isPalindrome) within a helper class without object instantiation.
- Optimized Primality Verification:** Utilizing primality checks, specifically skipping even numbers and multiples of 3, and iterating up to $\sqrt{n}$.
- Integer Reversal Algorithm:** Extracting digits dynamically using modulo and integer division operations to evaluate palindromic properties.
- Conditional Branching:** Categorizing numbers using logical conditions (AND / OR branching).

2. System Requirements

Component	Type	Minimum Specification	Recommended Specification
Hardware	Processor	Intel Core i3 / AMD Ryzen 3 (1.5 GHz)	Intel Core i5 / AMD Ryzen 5 (2.0 GHz+)
	RAM	1 GB	4 GB or higher
	Storage	200 MB free hard disk space	500 MB free SSD space
Software	Operating System	Windows 7 / Linux / macOS	Windows 10/11 or Ubuntu 20.04 LTS
	JDK Environment	Java Development Kit (JDK) 8	Java SE Development Kit 17 LTS or 21 LTS
	IDE / Editor	Command Line (javac / java)	Eclipse, IntelliJ IDEA, or VS Code

3. UML Class Diagram



4. Algorithm

Algorithm: Primality Check (`isPrime(n)`)

1. If $n \leq 1$, return false.
2. If $n \leq 3$, return true.
3. If n is divisible by 2 or 3, return false.
4. Loop variable i starting at 5 up to $i \leq n$, incrementing by 6 ($i = i + 6$):
 - If n is divisible by i or $i+2$, return false.
5. Return true if no factors were found.

Algorithm: Palindrome Check (`isPalindrome(n)`)

1. Store the original value of n in variable `original` and initialize `reversed = 0`.
2. While $n \neq 0$:
 - Extract the last digit: $digit = n \% 10$.
 - Append digit to reversed: $reversed = reversed * 10 + digit$.
 - Remove the last digit: $n = n / 10$.
3. Return true if `original == reversed`, else return false.

Algorithm: Main Program Flow (`LabProgram2`)

1. Initialize a Scanner instance to accept user input.
2. Prompt user to enter an integer n .
3. Call `primepalindrome.isPrime(n)` and store the result in boolean variable `prime`.
4. Call `primepalindrome.isPalindrome(n)` and store the result in boolean variable `palindrome`.
5. Evaluate logical conditions:
 - If `prime` AND `palindrome` are both true, print: *"n is a PRIME NUMBER AND PALINDROME"*.

- Else if prime is true, print: *"n IT IS PRIME BUT NOT PALINDROME"*.
- Else if palindrome is true, print: *"n IS NOT PRIME BUT IT IS A PALINDROME"*.
- Else, print: *"n IS NEITHER PRIME NOR BE PALINDROME"*.

6. Close the scanner object.

5. Compilation & Execution Instructions

Note: Save the source file with the exact name `LabProgram2.java` matching the public driver class name.

Step 1: Open Terminal / Command Prompt

Navigate to the source code folder:

```
cd path/to/your/java/files
```

Step 2: Compile the Program

```
javac LabProgram2.java
```

Generates class files `primepalindrome.class` and `LabProgram2.class` upon successful compilation.

Step 3: Execute the Program

```
java LabProgram2
```

6. Source Code

```
import java.util.Scanner;

class primepalindrome {
    static boolean isPrime(int n) { // method to check prime number
        if (n <= 1)
            return false;
        if (n <= 3)
            return true;
        if (n % 2 == 0 || n % 3 == 0)
            return false;
        for (int i = 5; (long) i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0)
                return false;
        }
        return true;
    }

    static boolean isPalindrome(int n) {
        int original = n;
        int reversed = 0;
        while (n != 0) {
            int digit = n % 10;
            reversed = reversed * 10 + digit;
            n /= 10;
        }
    }
}
```

```

    }
    return original == reversed;
}
}

public class LabProgram2 {
    public static void main(String[] args) {
        Scanner pp = new Scanner(System.in);
        System.out.println("enter the number");
        int n = pp.nextInt();
        boolean prime = primepalindrome.isPrime(n);
        boolean palindrome = primepalindrome.isPalindrome(n);
        if (prime && palindrome) {
            System.out.println(n + " is a PRIME NUMBER AND PALINDROME");
        } else if (prime) {
            System.out.println(n + " IT IS PRIME BUT NOT PALINDROME");
        } else if (palindrome) {
            System.out.println(n + " IS NOT PRIME BUT IT IS A PALINDROME");
        } else {
            System.out.println(n + " IS NEITHER PRIME NOR BE PALINDROME");
        }
        pp.close();

        /* Corrected typo inside prime check loop:
           Replaced 'n % 2 == 0' with 'n % i == 0' inside the for-loop */
    }
}

```

7. Program Execution & Output Samples

Case 1: Both Prime and Palindrome Input (e.g., 131)

```

enter the number
131
131 is a PRIME NUMBER AND PALINDROME

```

Case 2: Prime but Not Palindrome Input (e.g., 17)

```

enter the number
17
17 IT IS PRIME BUT NOT PALINDROME

```

Case 3: Palindrome but Not Prime Input (e.g., 121)

```

enter the number
121
121 IS NOT PRIME BUT IT IS A PALINDROME

```

Case 4: Neither Prime nor Palindrome Input (e.g., 20)

```
enter the number
20
20 IS NEITHER PRIME NOR BE PALINDROME
```

Code Analysis & Bug Fix Note: In the original source code loop inside `isPrime()`, the condition checked `n % 2 == 0` instead of `n % i == 0`. That redundant check was corrected to `n % i == 0` to properly implement the primality test algorithm.